PA4 - A-Frame VR Scene with Custom Component

Student: Islamiya Okassova Student ID: 242322 Variant: 2322

GitHub Link - https://github.com/islamyiaok-cpu/PA4-AFrame-VR

Theme: Underwater Research Base / Motion: Pulse / Interaction: Point-and-click / Layout: Six exhibits on a 4 m radius ring

Implementation: A-Frame 1.8.0, components.js, three image textures, a camera rig, and a gallery system.

E1 - Component lifecycle and runtime updates

Prediction: A-Frame will invoke init/update/play during initialization, update after attribute changes, pause/play on lifecycle actions, and remove when a component or entity is removed.

Result: Timestamped console logs showed init, update and play for exhibits. Changing amplitude to 0.4 triggered update; pause(), play(), and removeAttribute() produced the corresponding lifecycle logs. Removing exhibit-2 also triggered cleanup, and the gallery count changed to five.

Explanation: The custom component implements lifecycle hooks and time-delta-driven motion. Amplitude changes are smoothed rather than instantly applied. The system unregisters removed exhibits.

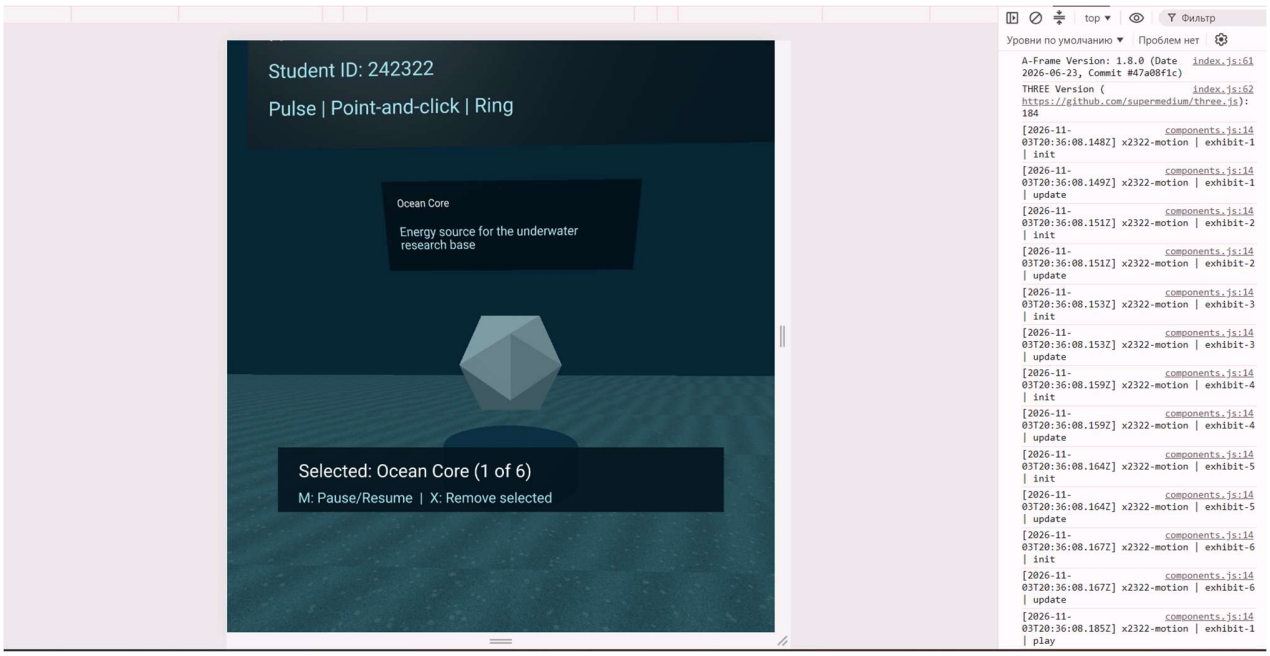


Figure 1. Initial motion lifecycle logs (init/update/play)

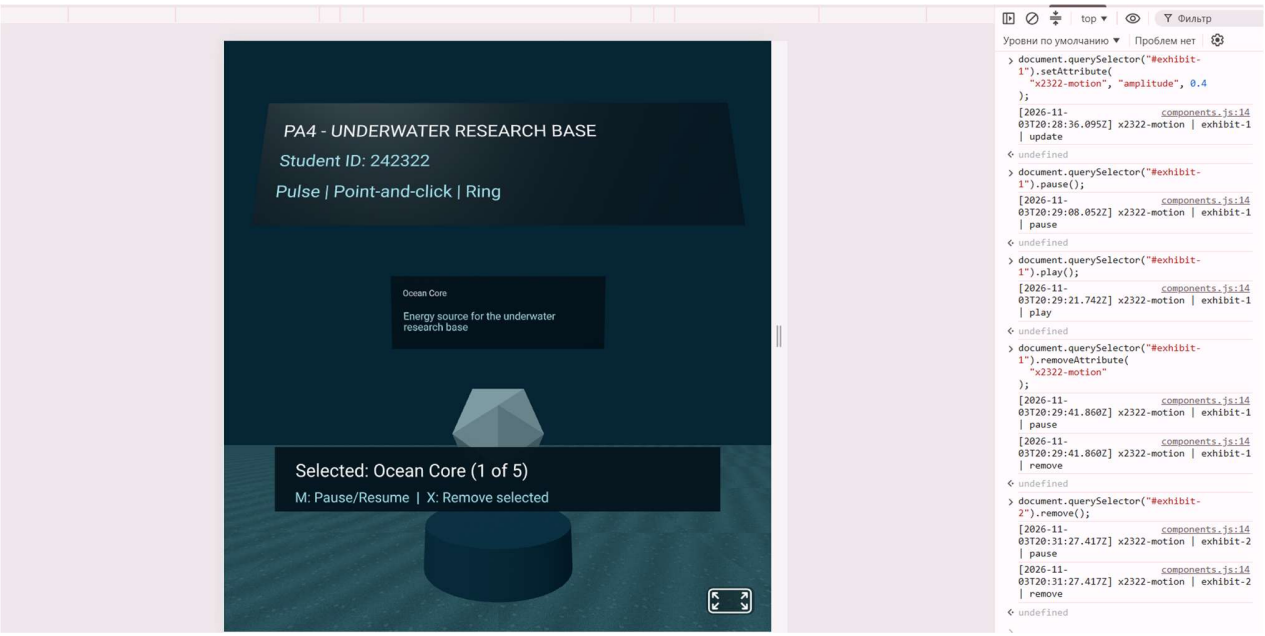


Figure 2. Runtime update, pause/play, removal and gallery count.

E2 - Chapter 8 loader versus corrected texture component

Prediction: The Chapter 8 example stores texture, material and mesh objects in this.data, although schema data should describe serializable component properties. Changing src may replace runtime objects; a cleaner implementation should store them on the component instance.

Result: In the Chapter 8 test, changing src from textures/coral.png to textures/metal.png updated the displayed texture. Console checks returned true for texture, material and mesh before and after. In the project component, schema data contained src, repeat, normalSrc and anisotropy, while runtime fields held the THREE objects. A live ground-repeat change to 15 x 15 was reflected in the GPU texture.

Explanation: The Chapter 8 example can render, but putting mutable THREE instances into this.data mixes configuration with runtime resources. The project keeps textures/materials on this and disposes of them during replacement/removal.

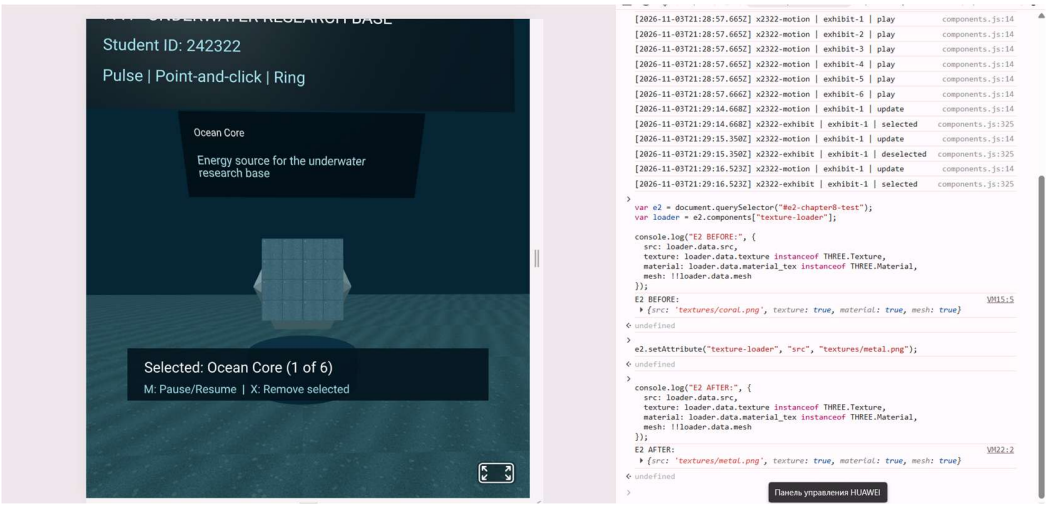


Figure 3. Chapter 8 loader: before/after source change.

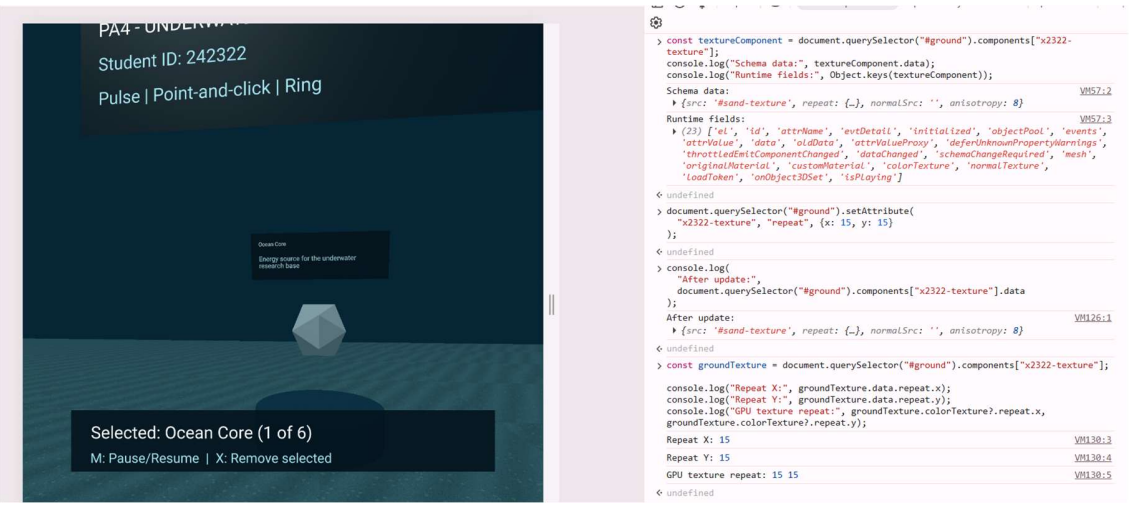


Figure 4. Schema versus runtime fields and live repeat update.

E3 - Texture filtering comparison

Prediction: Nearest filtering will appear sharper/pixelated, trilinear mipmapping smoother at a distance, and anisotropic filtering should improve oblique-angle texture detail.

Result: At a grazing view of the seafloor, the Chapter 8 setting used min/mag = 1003/1003 and anisotropy 1. Three.js defaults used 1008/1006 and anisotropy 1. The chosen setting used 1008/1006 and anisotropy 8. The screenshots show the visible texture differences, though the scene and capture conditions limit subjective comparison.

Explanation: 1003 is NearestFilter; 1008 is LinearMipmapLinearFilter; 1006 is LinearFilter. Mipmaps reduce aliasing with distance, while anisotropy can preserve detail at grazing angles.

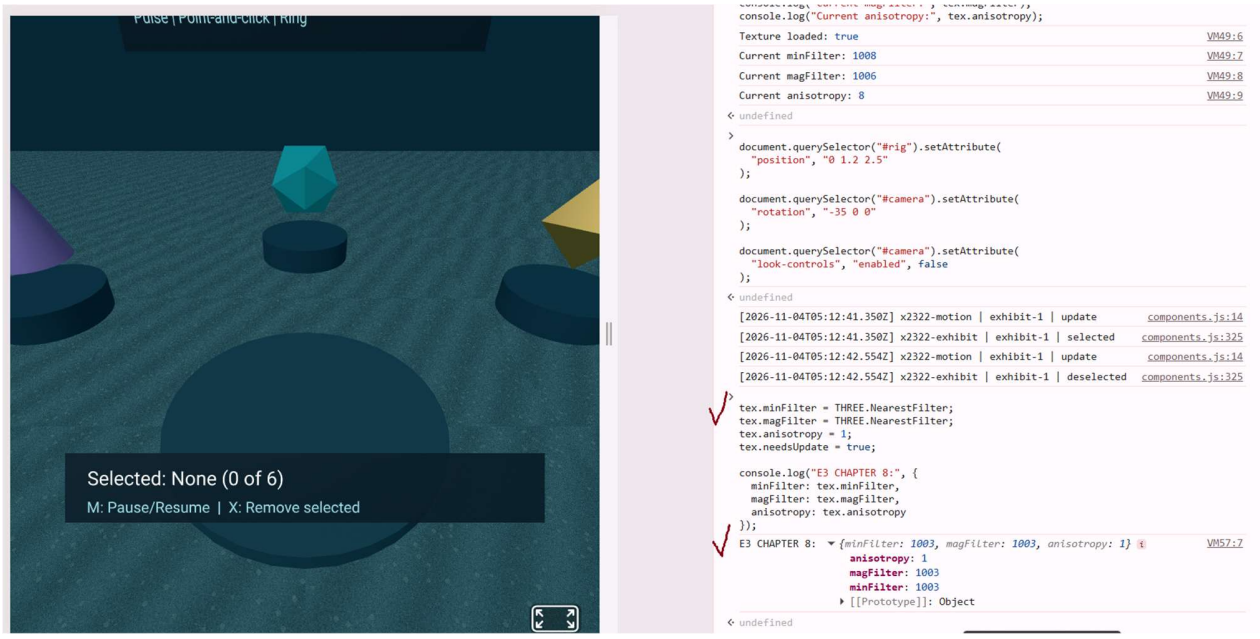


Figure 5. Nearest filtering: 1003/1003, anisotropy 1

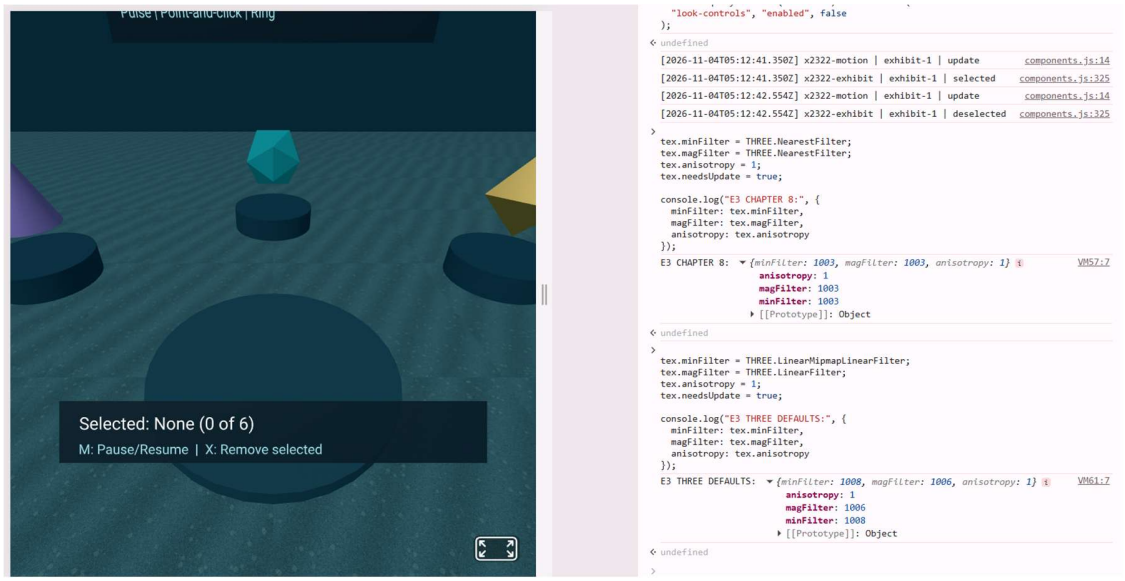


Figure 6. Three.js default filters: 1008/1006, anisotropy 1

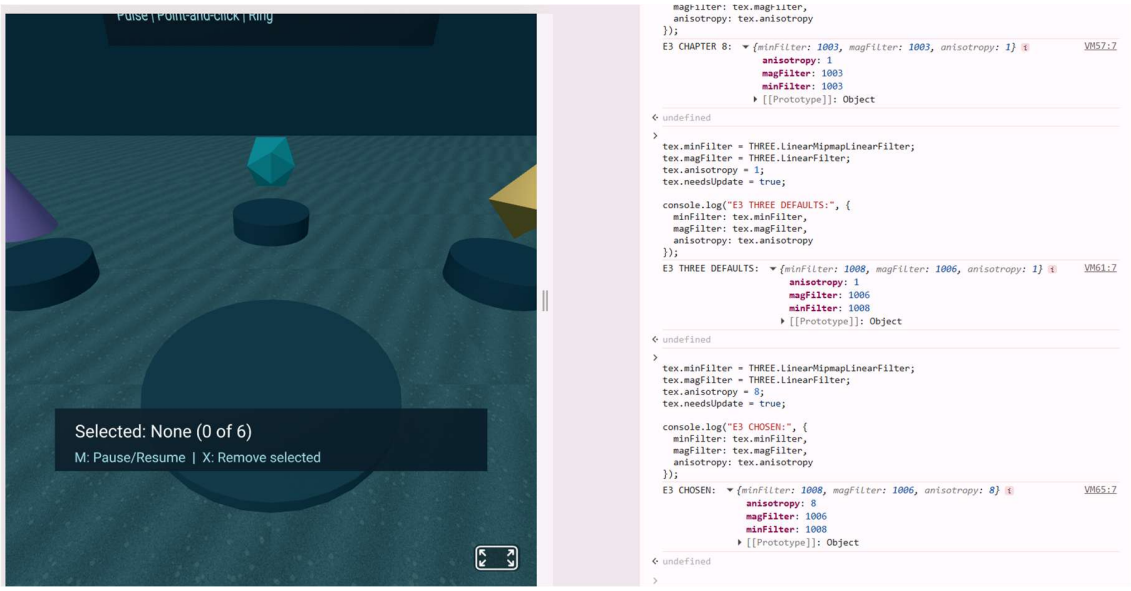


Figure 7. Chosen filters: 1008/1006, anisotropy 8

E4 - A-Frame architecture compared with PA3

Prediction: A-Frame will expose a Three.js renderer, Scene, PerspectiveCamera and WebXR manager without requiring manual creation of each object.

Result: Console inspection confirmed sceneEl.renderer (WebGLRenderer), sceneEl.object3D (Scene), sceneEl.camera (PerspectiveCamera) and sceneEl.renderer.xr (WebXR manager). XR enabled was false in the desktop inspection.

Explanation: In PA3, the Three.js scene, renderer and XR session were handled explicitly. A-Frame declarative entities/components provide higher-level scene management while retaining access to Three.js internals.

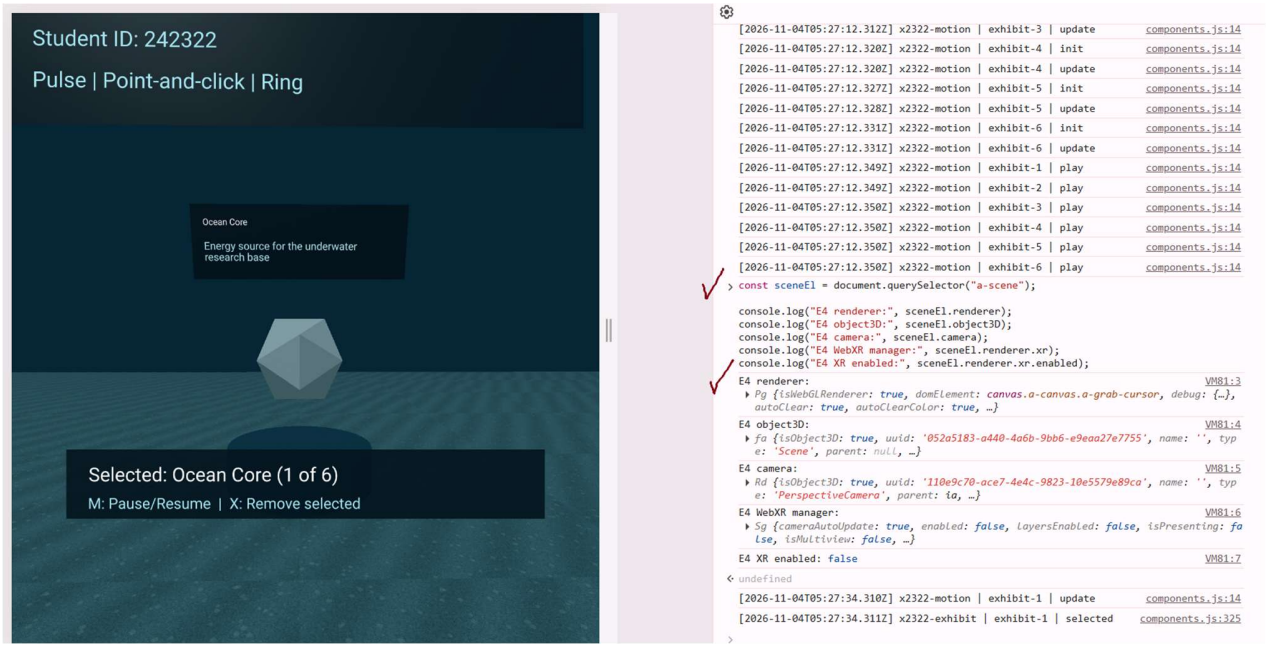


Figure 8. Renderer, scene, camera and WebXR manager inspection

E5 - Performance and optimization

Prediction: Adding 60 separate cubes will increase rendering work; replacing them with one InstancedMesh should reduce draw calls, although FPS may not improve in a lightweight scene.

Result: The baseline measured 61 FPS, 16 draw calls, 7,078 triangles, 17 geometries and 3 textures. With 60 cubes: 61 FPS, 29 calls, 7,234 triangles, 18 geometries, 3 textures. Instancing: 59 FPS, 17 calls, 7,798 triangles, 18 geometries, 3 textures.

Explanation: Instancing reduced draw calls by 12 (about 41.4%) relative to the separate-cube test. Triangles increased, and the measured FPS was slightly lower, so these measurements support draw-call reduction, not an FPS improvement. Visibility/culling and shared resources help explain why 60 extra entities did not add 60 draw calls.

Metric	Baseline	60 cubes	Instanced
FPS	61	61	59
Draw calls	16	29	17
Triangles	7,078	7,234	7,798
Geometries	17	18	18
Textures	3	3	3

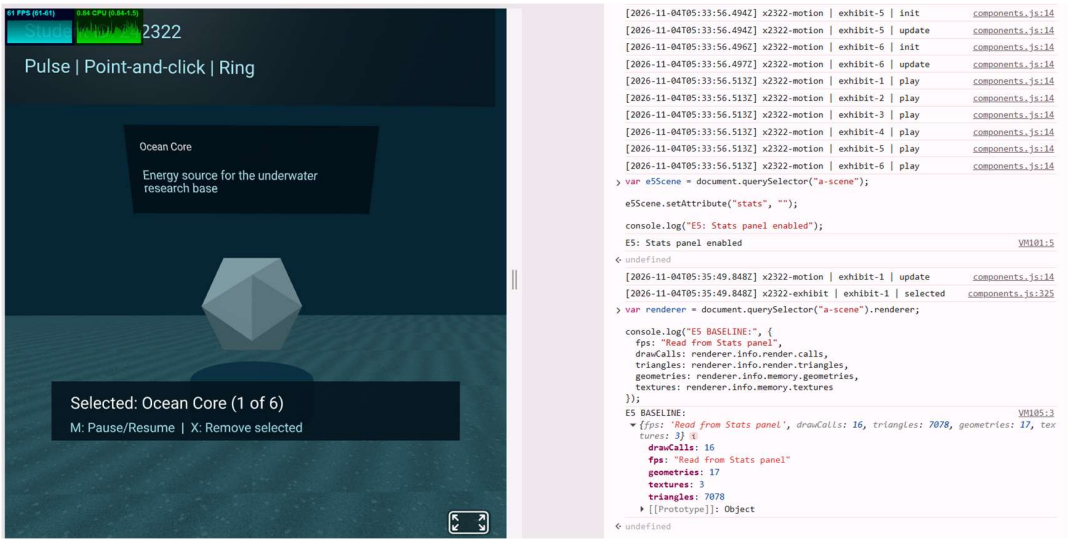


Figure 9. Baseline counters

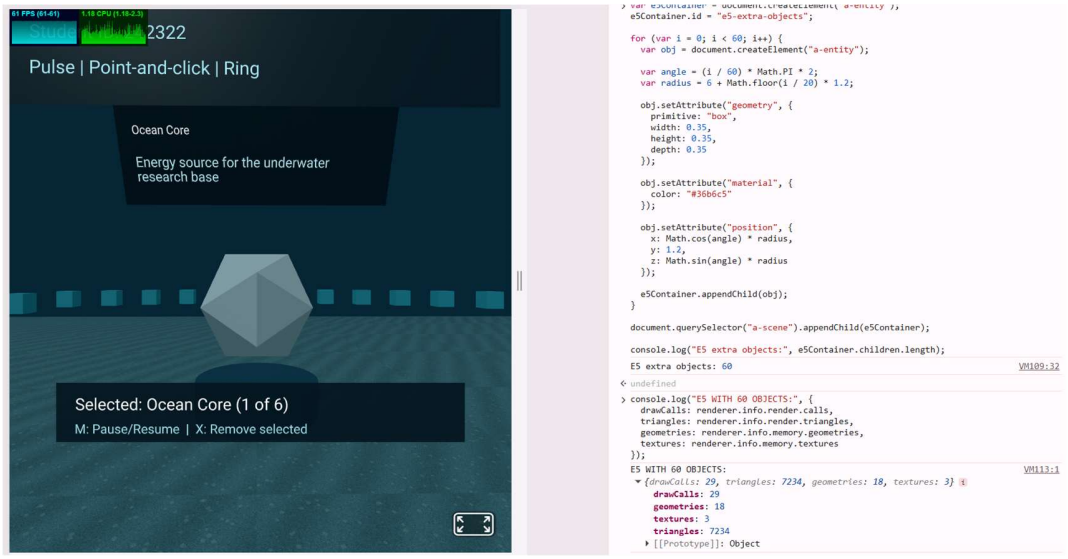


Figure 10. Sixty separate entities and counters

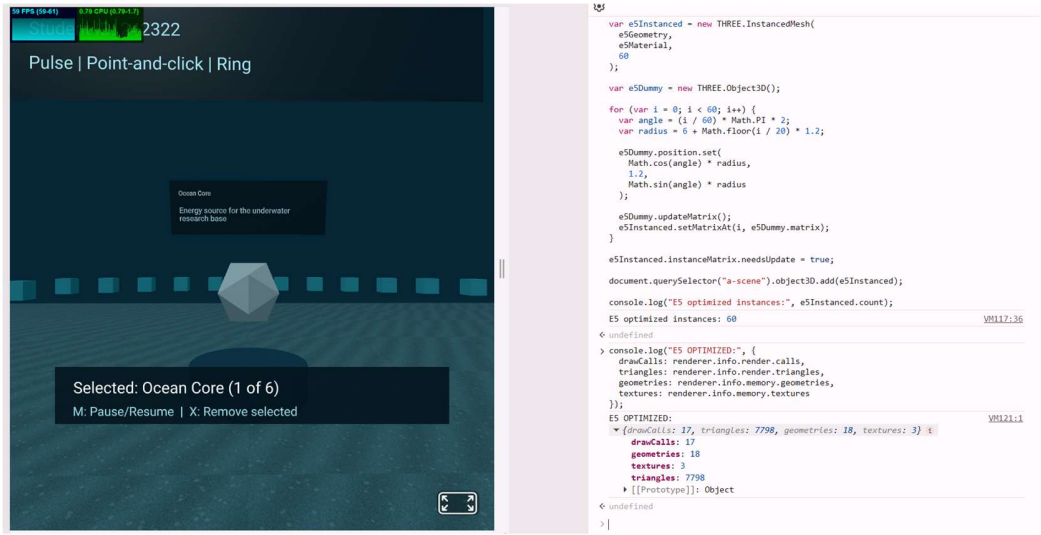


Figure 11. Sixty instanced cubes and optimized counters

E6 - VR ergonomics (design rationale)

Prediction: A ring of six exhibits with a 4 m radius should provide enough space for navigation and observation. A camera height of 1.6 m should offer a natural standing viewpoint.

Result: During desktop testing, W/A/S/D movement and mouse rotation worked smoothly. Point-and-click selection changed the exhibit's appearance and displayed its information panel. The text was readable without approaching the exhibit closely. However, the panel also changed size during the pulsing animation.

Explanation: The circular layout distributes exhibits around the visitor and supports exploration. Visual highlighting makes selection easier to recognize. However, pulsing information panels may reduce reading comfort, especially at longer distances. Larger, stationary panels could improve accessibility.

Development log

Issue: Runtime textures/materials must not be stored inside A-Frame schema data, as in the Chapter 8 example.

Detection: E2 console inspection compared the Chapter 8 loader with x2322-texture schema and runtime fields.

Fix: Keep THREE references on the component instance, update texture repeat live, and dispose of replaced resources.

Relevant commit: 30e5d39 - Implement texture loader and gallery system